

Project 1: This Ship is on Fiiiiiire!

Your Name (NetID)

1. Implementation and Bot 4 (Question 1)

I wrote everything in Python and kept it modular, since all four bots are really the same thing with a different way of choosing a path. The ship lives in `ship.py`, the fire in `fire.py`, the search algorithms in `planning.py`, the four path-choosing strategies in `planners.py`, a single Bot class in `bots.py`, the simulation loop in `simulation.py`, and the experiment and analysis scripts in `experiments.py`, `analyze.py` and `figures.py`.

As in my original plan, the ship is a $D \times D$ grid of Tile objects, and each tile tracks whether it is open or blocked, whether it is on fire, whether the bot or the button is on it, and a list of its open neighbours. Walls never change after generation, so each tile's neighbour list is computed once and every search just walks it. Figure 1 shows one ship after each of the two generation phases. Phase 1 produces a tree, because every cell it opens touches exactly one open cell at that moment, so there is exactly one route between any two cells. The loops that phase 2 adds matter a lot for this project: without them there would only ever be one route to the button, and the bots would have nothing to decide. Rescanning the whole grid for candidate cells on every iteration of phase 1 would cost $O(D^2)$ per opening, so $O(D^4)$ overall. Instead I keep a list of the current candidates and update it as I go: opening a cell only changes the neighbour counts of its four neighbours, so at most four cells join or leave the list, and a dictionary from position to list index lets me remove one in constant time. That makes phase 1 $O(D^2)$, and phase 2 similarly only rechecks the cells around each opening. At $D = 50$ a ship has about 1,631 open cells (65% of the grid), phase 1 leaves about 449 dead ends, phase 2 cuts that to about 224, and generating a ship takes about 19 ms.

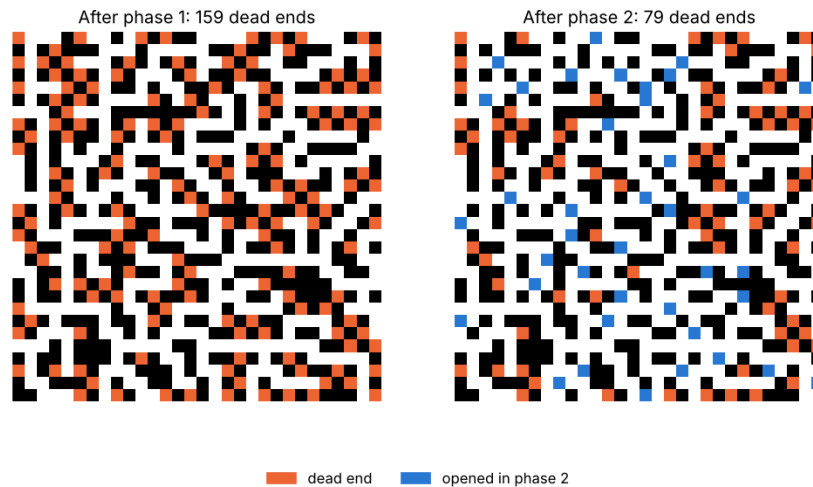


Figure 1: The same 30 by 30 ship after phase 1 (a tree) and after phase 2. Orange cells are dead ends; blue cells were opened in phase 2 and create the loops.

Following the TA's feedback, the update is synchronous: I first go over every cell and decide which ones catch fire, counting K from the fire as it was at the start of the step, and only then set them all on fire, so a newly ignited cell can't spread the fire again in the same step. I draw exactly one random number per cell per update, in row order, even for walls and burning cells, so a given random generator always produces exactly the same fire. Each trial stores its generator, and every bot run on that trial starts the fire from a fresh copy, so all four bots face the identical fire progression. That makes every comparison between bots a paired one, which turned out to be worth a lot of data (Section 2). I label every failure with its cause: the bot walked into fire, the fire spread onto the bot, the button burned before the bot got there, or the bot got cut off with every route to the button running through fire. The last two end the trial early, which changes nothing since the fire never goes out.

All of the searches follow the pseudocode from lecture, with a fringe of states still to explore, a closed set of explored states, and a prev map recording which state each child was reached from (with `prev[start] = None`), which is followed backwards from the goal to rebuild the path. BFS is GraphSearch with a queue as the fringe, so the first time the goal comes off the fringe it was reached along a shortest path, and the cells the bot wants to avoid are the restricted states. I added one line to the lecture version: a child already in prev is already on the fringe, so it isn't added a second time. Without that line a cell's prev gets overwritten by later parents, which changes which of several equally short paths BFS returns. A bot is then just a name, a planner function that returns a path to the button, and a flag for whether to call the planner again every step. Bots 1 and 2 share the same planner (BFS avoiding the burning cells); Bot 1 calls it once at the start and Bot 2 every step. Bot 3's planner avoids burning cells and their neighbours, and when no such path exists it simply calls Bot 2's planner.

Bot 2 avoids where the fire is. But the bot doesn't really care where the fire is right now; it cares whether the fire will be on a cell *when the bot gets there*. A cell three steps from the fire is harmless if the bot crosses it on its next move, and deadly if the bot would only reach it twenty moves from now. So Bot 4 treats every cell as a race between the bot and the fire, estimates the probability that the fire wins that race, treats cells the fire will probably win as expensive, and then plans the cheapest route to the button with A*.

Every step, for every open cell c , Bot 4 computes two distances with BFS. The first, $k(c)$, is the number of moves the bot needs to reach c , avoiding the cells that are burning now. The second, $d(c)$, is how many cells the fire has to travel to reach c ; this is a BFS through the open cells (the fire can't cross walls) that starts with every burning cell on the fringe at once. Along a single route, the front of the fire advances one cell per step with probability q , because the next cell on the route has exactly one burning neighbour and so ignites with probability $1 - (1 - q)^1 = q$, independently each step. After k steps the front has therefore advanced $A_k \sim \text{Binomial}(k, q)$ cells, and the probability that the fire gets to c before the bot is

$$P(A_{k(c)} \geq d(c)) = \sum_{j=d(c)}^{k(c)} \binom{k(c)}{j} q^j (1 - q)^{k(c)-j}.$$

For the button I use $k - 1$ instead of k , because the bot presses the button before the fire gets to move. If this probability is above 0.6, the cell counts as "predicted fire". For example, at $q = 0.3$ a cell five cells from the fire that the bot would only reach in 20 steps is predicted fire (the fire wins that race 76% of the time), while a cell right next to the fire that the bot reaches in one step is not (30%). Bot 4 never actually sums the binomial during a trial. Since $P(A_k \geq d)$ only gets smaller as d grows, for each k there is a largest fire distance that still counts as dangerous, and I precompute that "danger radius" for every k once per value of q (updating the binomial distribution one step at a time) and keep it in a dictionary. Checking a cell during the trial is then a single comparison, $d(c) \leq \text{radius}[k(c)]$. Figure 2 plots the radius. The further away a cell is for the bot, the further from the fire it can be and still count as predicted fire, and the faster the fire, the steeper that growth.

Once it knows which cells are predicted fire, Bot 4 runs A* from its position to the button, where stepping onto a cell costs infinity if it is burning (so it is never entered), $1 + 20 = 21$ if it is predicted fire, and 1 otherwise. The heuristic is the Manhattan distance from a cell to the button. That is exactly the cost of a relaxed version of the problem (the same grid with every wall removed), so it never overestimates the true remaining cost, which makes it admissible. It is also consistent: every step costs at least 1 and the Manhattan distance changes by at most 1 per step, so $h(n) \leq C^*(n, n') + h(n')$ for any neighbour n' . As in the lecture notes, A* here is uniform cost search with the priority changed from $g(n)$ to $f(n) = g(n) + h(n)$, where $g(n)$ is the cheapest cost found so far from the bot to n . The fringe is a priority queue (a binary heap) that starts with just the bot's cell at priority $h(\text{start})$, with `dist[start] = 0` and `prev[start] = None`. Each iteration removes the cell `curr` with the smallest priority. If it is the button, A* is done: because h is consistent, the first time the button comes off the fringe its path is the cheapest one. Otherwise, for every open neighbour child that isn't burning, it computes `dist_to_child = dist[curr] + cost(child)`. If the child has never been seen, or this is cheaper than its current `dist`, it records the new `dist[child]`, sets `prev[child] = curr`, and adds the child to the fringe with priority `dist[child]` plus the child's Manhattan distance to the button. Like the A* in the notes, there is no closed list. Python's heap has no "add or update" operation, so when a child's distance improves it is simply added again; the old copy has a higher priority, and when it eventually comes off it can't improve anything, so it changes nothing. Ties in priority are broken by g and then by position,

so the search always returns the same path. When the button comes off, following `prev` back from it gives the plan, Bot 4 takes the plan's first step, and the whole thing (both BFSs, the predicted set and A^*) runs again the next step with the new fire. If A^* empties the fringe without reaching the button, every route runs through burning cells and the bot is cut off.

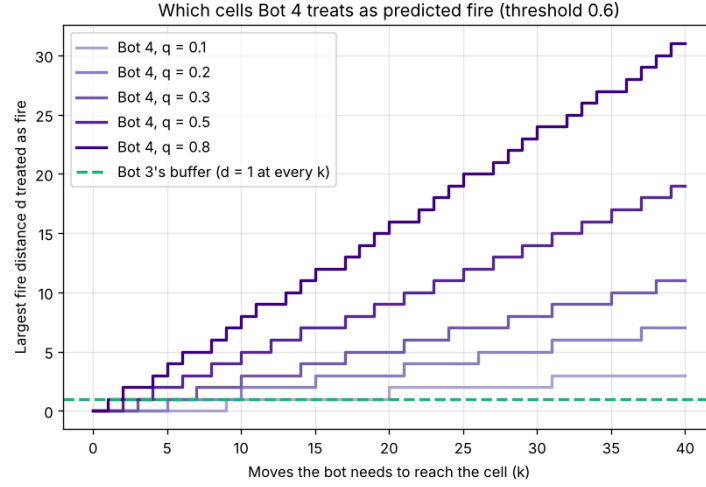


Figure 2: For a cell the bot needs k moves to reach, the largest fire distance d at which Bot 4 treats it as predicted fire, for several values of q . Bot 3's buffer is $d = 1$ no matter how far away the cell is.

Since a plan costs its length plus 20 for every predicted-fire cell on it, Bot 4 will take a detour of up to 20 extra steps to avoid one predicted-fire cell, but it goes straight through predicted fire when every way around is longer than that (or runs through predicted fire too). So the shortest path isn't avoided for being near the fire, only for being somewhere the fire will probably get first. Figure 3 shows a real decision. On its first move in trial 463 at $q = 0.3$, the shortest path (Bot 2's choice) is 25 steps but crosses 3 predicted-fire cells, for a cost of $25 + 3 \times 20 = 85$, while Bot 4's plan is 2 steps longer and crosses none, for a cost of 27. Two cells that are both 4 steps from the fire are treated differently: cell (43, 40), which the bot would reach in 15 moves, is flagged because the fire covers 4 cells within 15 steps 70% of the time, while (40, 41), which the bot could reach in 11, is not (43%). Figure 4 shows how the trial ended for all four bots.

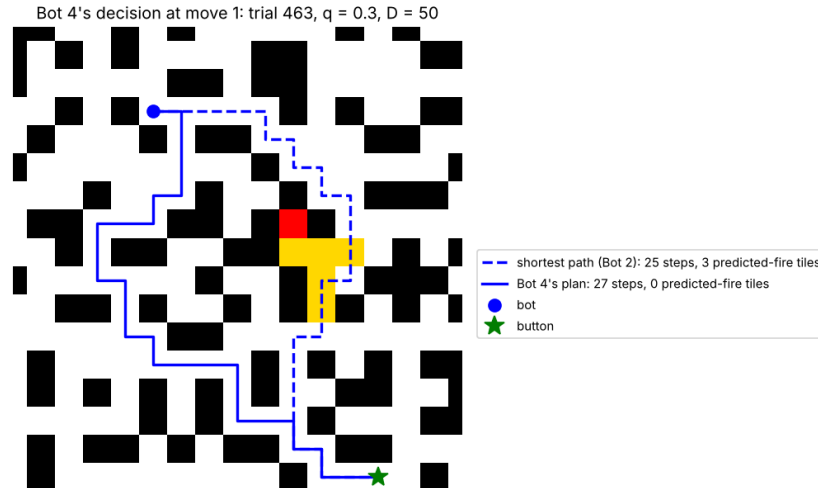


Figure 3: Bot 4's first decision on trial 463 ($q = 0.3$). Black cells are walls, red is burning and yellow is predicted fire. Bot 2's shortest path (dashed) is 25 steps through 3 predicted-fire cells; Bot 4's plan (solid) is 27 steps through none.



Figure 4: Trial 463 ($q = 0.3$) for all four bots, with each bot's path in blue and every cell that burned before its run ended in orange. Bot 4 took its detour and won in 27 steps. Bot 2 kept to the short route and was cut off after 24 steps, Bot 3 detoured by a different route and was cut off after 28, and Bot 1 walked into the fire.

Bot 4 uses everything the bot can observe: the layout, which cells are burning right now, its own position, the button's position, and q (I assume the bot knows q). From those it builds two distance maps, turns them into a prediction of where the fire will beat the bot, and weighs route length against how much predicted fire a route crosses. Because it replans every step, each new fire cell immediately changes both maps, the prediction and the plan. Its avoided region differs from Bot 3's buffer in three ways. It depends on where the bot is: a cell right next to the fire that the bot can reach first isn't avoided, while a cell far from the fire that the bot would only reach much later can be (Figures 2 and 3). It depends on q : a slow fire predicts a thin band or nothing, a fast fire a wide one. And it is soft: a predicted-fire cell is a cost to weigh against detour length, not a wall.

My original idea was A* with a Manhattan heuristic "between the fire and the bot", with burning cells costing extra and open cells that have a high (over 60%) chance of catching fire treated as burning. Three things changed when I tried to pin it down. First, an A* heuristic has to estimate the remaining cost to the goal, so it became the Manhattan distance to the button; the fire-versus-bot comparison moved into the race instead. Second, read literally, "a cell with more than a 60% chance of catching fire" means $1 - (1 - q)^K > 0.6$ for the next step. Only cells touching the fire have $K \geq 1$, so that rule can only ever flag cells inside Bot 3's buffer (and with one burning neighbour, only when $q > 0.6$), which makes it Bot 3 with a thinner buffer, and at low q it would never flag anything and would just be Bot 2. The race asks the same question, "will this cell be on fire?", but at the time that matters, when the bot would actually get there. I kept the literal rule as a variant to test it, and it did no better than Bot 2. Third, the fire can only spread through open cells, so $d(c)$ is measured through the maze rather than with the Manhattan distance (the Manhattan version was slightly worse in tuning, within the noise, and slower). My plan also said the heuristic would be the Euclidean distance to the button; since the bot only moves up, down, left and right, the Manhattan distance is the exact cost on an open grid, it is still admissible, and because it is never smaller than the Euclidean distance it guides the search better.

Each burning cell gives each unburnt neighbour its own chance q every step, and the real fire reaches a cell at the earliest time over *all* routes, while the race model only follows one shortest route. On a tree-shaped region with one burning cell there is only one route, so the formula is exact. Everywhere else, extra routes (the loops from phase 2, or several burning cells) can

only make the real fire faster, so the true probability is never lower than the model's, and the model is a slightly optimistic lower bound. I checked both of these with unit tests against simulated fires. There is also one known limitation: $k(c)$ is the bot's *shortest* distance to c , not the step at which its plan actually reaches c . On a long detour the bot reaches cells (and the button) later than $k(c)$, so those cells are more dangerous than the model thinks and detours look safer than they are. Section 3 shows this losing a trial.

I wrote the code to be readable first (a grid of objects, (row, col) tuples and plain loops), and made it fast where it counted. Ship generation is $O(D^2)$ instead of $O(D^4)$. Each tile's open neighbours are computed once. The binomial tail is turned into a lookup table once per q , so the danger check for a cell is one comparison. The Manhattan heuristic keeps A* searching towards the button instead of in every direction, as plain uniform cost search would. Trials stop as soon as the outcome is certain. Because every bot sees the same fire, comparisons are paired, which needs about 16 times fewer trials for the same precision on a difference. And trials run in parallel on every core, with each trial's random seed depending only on (seed, q , trial number), so the results don't depend on how the work is split up and any single trial can be replayed. Table 1 shows how long each bot spends deciding. A whole trial with all four bots takes about 0.7 s on one core, so the main run of 83,000 trials took several hours on 4 cores.

Table 1: Mean time each bot spends deciding at $D = 50$ (175 trials, q from 0.1 to 0.7).

	Bot 1	Bot 2	Bot 3	Bot 4
Per move	23 μ s	514 μ s	584 μ s	3.0 ms
Per trial	0.8 ms	20 ms	23 ms	120 ms
Work per move	one BFS at the start, then none	one BFS	one or two BFSs	two BFSs, a pass over the cells, A*

2. Experiments and results (Question 2)

For the main experiment I used $D = 50$, which kept the full run to several hours on four cores, and generated a fresh ship for every trial. I first ran 1,000 trials at every q from 0 to 1 in steps of 0.05. That showed that below $q \approx 0.1$ Bots 2-4 win about 98% of trials or more and are within a few tenths of a point of each other, and that above $q \approx 0.75$ all four bots are within a point of each other. So the interesting range is roughly 0.1 to 0.7, and that is where I put most of the data: every q from 0.1 to 0.7 in steps of 0.025 got 3,000 trials. In total that is 83,000 trials, with all four bots run on every one of them. With 3,000 trials, each success rate has a 95% confidence interval of about ± 1.6 points, which on its own isn't enough to separate bots that differ by 1-2 points. But because the bots share each trial's fire, I can measure the *difference* between two bots trial by trial: at $q = 0.4$, Bot 4 minus Bot 2 is $+1.6 \pm 0.5$ points paired, where two independent samples of the same size would give ± 2.0 . Matching that precision with independent samples would take about 16 times as many trials. I tuned Bot 4 on a separate set of trials with a different seed (Section 3), so the main results were never used to choose its settings, and I spot-checked two other ship sizes.

Table 2: Success rates and certain wins at selected q .

q	Bot 1	Bot 2	Bot 3	Bot 4	Certain wins
0	100.0%	100.0%	100.0%	100.0%	47.8%
0.1	94.9%	98.1%	98.2%	98.1%	49.7%
0.2	89.4%	93.1%	93.4%	93.3%	49.5%
0.3	83.1%	86.6%	87.0%	87.4%	48.9%
0.4	77.3%	79.3%	79.9%	80.9%	50.5%
0.5	74.0%	74.8%	75.3%	75.8%	51.8%
0.6	67.3%	67.6%	67.6%	68.4%	49.5%
0.7	61.0%	61.2%	61.1%	61.5%	49.2%
0.8	56.2%	56.3%	56.2%	56.3%	49.5%
0.9	50.8%	50.8%	50.8%	50.6%	47.3%
1	51.7%	51.7%	51.7%	51.7%	51.7%

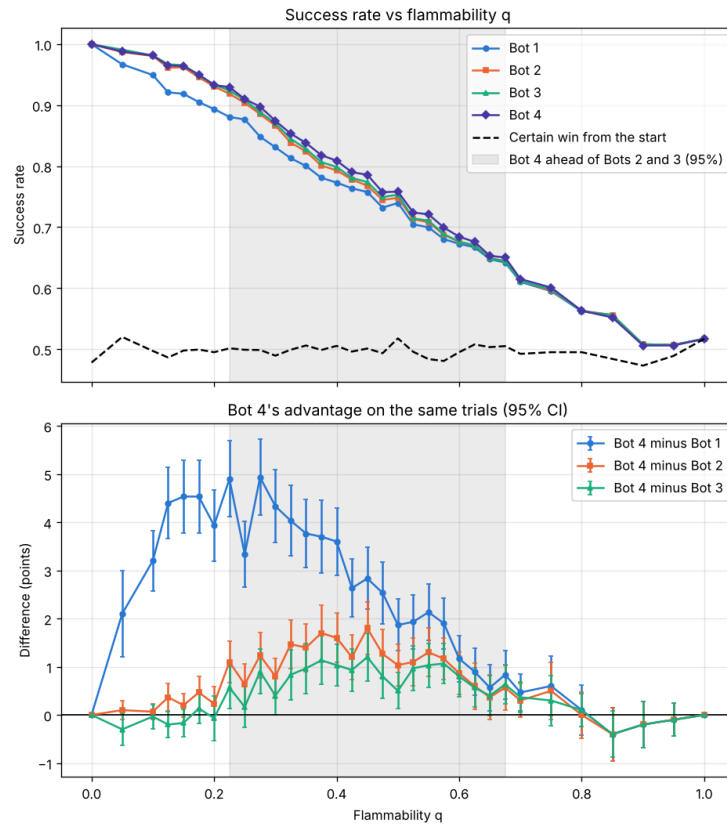


Figure 5: Top: success rate against flammability q for every bot, with the share of trials that are certain wins from the start dashed. Bottom: Bot 4's success rate minus each other bot's on the same trials, in percentage points, with 95% confidence intervals. The shaded band is where Bot 4 beats both Bot 2 and Bot 3 with 95% confidence.

Figure 5 is the centerpiece, and Table 2 has the numbers behind it. For small q all the bots are equally good: at $q = 0$ every bot wins every trial, and up to $q = 0.2$ Bots 2-4 are within 0.6 points of each other. Bot 1 is the exception, because it never replans, so even a slow fire that creeps onto its path kills it (96.7% at $q = 0.05$ against about 99% for the others). For large q all the bots are equally bad: from $q = 0.8$ up they are within 0.6 points of each other, and at $q = 1$ every bot wins exactly 51.7% of trials, because the fire moves as fast as the bot and the outcome only depends on whether the bot started ahead of it. In between, there's where you can see Bot 4 outperforming the other three: from $q = 0.225$ to 0.675 (the shaded band), Bot 4

beats both Bot 2 and Bot 3 with 95% confidence at 17 of the 19 values of q I tested (at 0.25 and 0.65 it is ahead but within the noise). Its lead peaks at $+1.8 \pm 0.5$ points over Bot 2 and $+1.2 \pm 0.5$ over Bot 3 at $q = 0.45$ (78.6% against 76.8% and 77.4%), and it is 3 to 5 points ahead of Bot 1 for q between 0.125 and 0.4.

Those differences look small next to the overall drop with q , and the certain-win check explains why. Before simulating anything, I check whether the bot has a path that even the fastest possible fire ($q = 1$) can't catch. The fire moves at most one cell per step whatever q is, so a BFS from the fire cell gives the earliest step at which the fire could possibly reach each cell. A second BFS, one move at a time, then looks for a path on which the bot enters every cell strictly before that (and reaches the button no later than that, since the button is pressed before the fire moves). Arriving anywhere earlier is never worse, so each cell only has to be considered once. If such a "fireproof" path exists, the trial is a certain win for any bot that follows it, whatever the fire does, and finding out costs two BFSs instead of a whole simulation. It turns out that 47-52% of trials are certain wins at *every* q (the check doesn't depend on q), and every bot won every single one of them: 41,283 out of 41,283 for each bot, even though none of the bots actually looks for a fireproof path. So a data-collection run could count these trials as wins without simulating them, roughly halving the work (I simulated them anyway to confirm it). It also explains the shape of Figure 5: success can never drop below the dashed line, which is why the curves level off near 50% at high q instead of falling to zero, and only the other half of the trials can tell the bots apart.

Bot 4 does not outperform everywhere. At $q \leq 0.2$ it ties Bots 2 and 3, and Bot 3 is even up to 0.3 points ahead at $q = 0.05$ -0.15 (within the noise). A slow fire gives the cells next to it only moderate risks: at $q = 0.2$, a cell next to the fire that the bot reaches within 4 steps has a 20-59% chance of burning first. That is under Bot 4's 60% threshold, so Bot 4 walks right past the fire, while Bot 3's buffer avoids exactly those cells. Of the 69 trials at $q \leq 0.2$ that Bot 4 lost and Bot 3 won, 65 were the fire spreading onto the bot. At very high q (0.85-0.95) Bot 4 is 0.1-0.4 points behind the others, again within the noise, because of detours that look safe to the race model but aren't (Section 3).

I also ran Bot 4 on smaller and larger ships: 2,000 trials per q at $D = 25$ and 400 per q at $D = 100$ (Figure 6). Success at a given q rises a little with D , but the pattern is the same at every size: about half the trials are certain wins, Bot 4 ties Bot 3 at low q , and it leads Bot 3 by 0.4-1.1 points in the middle range at $D = 25$ and 50 (at $D = 100$ the estimates are positive but too noisy to be significant). Bot 4's decision time grows quickly with D , since each move costs $O(D^2)$ and trials get longer; a trial takes about 16 times as long at $D = 100$ as at $D = 50$.

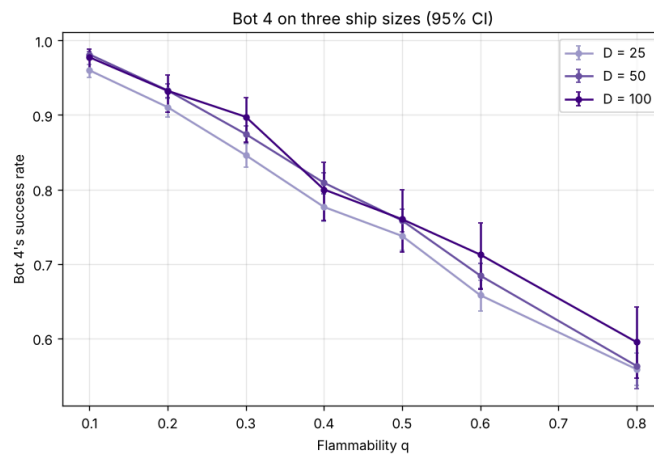


Figure 6: Bot 4's success rate on ships of size 25, 50 and 100, with 95% confidence intervals (wider at $D = 100$, which had fewer trials).

To check whether Bot 4 really decides differently from Bot 2, I count every move that Bot 2's rule could not have made, meaning a move that doesn't step along *some* shortest path to the button through cells that aren't burning. This doesn't depend on how ties between equally short paths are broken, so a nonzero count means a genuinely different decision, and Bot 2's own count is always zero. Because all the bots face the same fire, I can then compare their outcomes trial by trial (Table

3). Bot 4 does mostly agree with Bot 2: in 89% of trials every one of its moves is one Bot 2 could have made. That makes sense, because when the fire is far away or slow, the race predicts nothing on the shortest path and the cheapest A* path is a shortest path. But in the 10.9% of trials where Bot 4 does leave Bot 2's rule, its departures help 3.5 times as often as they hurt. Bot 3 departs about as often, but its departures help only 1.6 times as often as they hurt, because its buffer is a fixed rule that doesn't ask whether the detour is worth it. Even without ever leaving Bot 2's rule, Bot 4 still wins 233 trials that Bot 2 loses (and loses 50): when there are several equally short paths, the penalty steers it to the one with fewer predicted-fire cells, where Bot 2 just takes whichever one BFS finds first. Figure 7 shows how often each bot departs from Bot 2's rule as q grows.

Table 3: Outcomes against Bot 2 on the same 83,000 trials, split by whether the bot ever made a move Bot 2's rule couldn't have made.

Bot	Leaves Bot 2's rule?	Trials	Won where Bot 2 lost	Lost where Bot 2 won
Bot 3	yes	8,699 (10.5%)	466	299
Bot 3	no	74,301	75	0
Bot 4	yes	9,060 (10.9%)	698	197
Bot 4	no	73,940	233	50

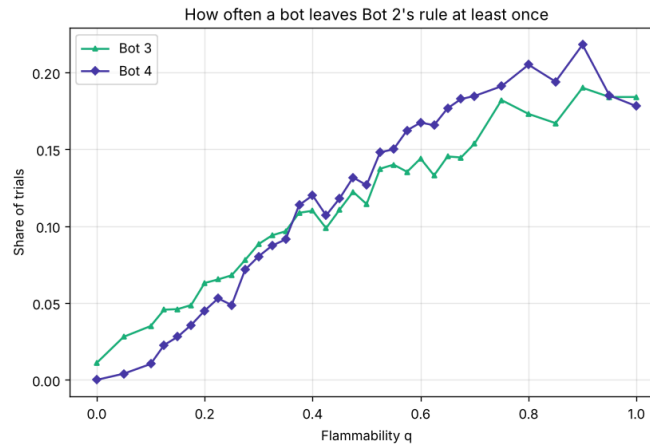


Figure 7: Share of trials in which Bot 3 or Bot 4 makes at least one move that Bot 2's rule (step along some shortest fire-free path) could not have made.

3. Failures, process and the ideal bot (Questions 3 and 4)

Table 4 and Figure 8 show how each bot fails. Bot 1 mostly dies by walking into fire: it never looks again after planning, so it follows its plan straight into fire that has spread across it, and it is almost never cut off because it never checks. Bot 2 never walks into fire, but it hugs the fire's edge, and 14% of its failures are the fire spreading onto it, the most of Bots 2-4; it also heads down routes the fire is about to close and then gets cut off. Bot 3's buffer brings "fire spread onto bot" down to 5% of its failures, but its detours cost time, so more of its failures are the button burning first or being cut off. Bot 4 sits in between on "fire spread onto bot" (7%), since it keeps its distance from the fire only when the race says the fire will win. For Bots 2-4, nearly all failures are the button burning or the bot being cut off: the fire got to the button, or across every route to it, before the bot could.

Table 4: How each bot's failures break down, pooled over every q .

Bot	Failures	Walked into fire	Fire spread onto bot	Button burned first	Cut off from button
Bot 1	19,262	40%	25%	36%	0%
Bot 2	17,736	0%	14%	40%	45%
Bot 3	17,494	0%	5%	45%	50%
Bot 4	17,052	0%	7%	46%	47%

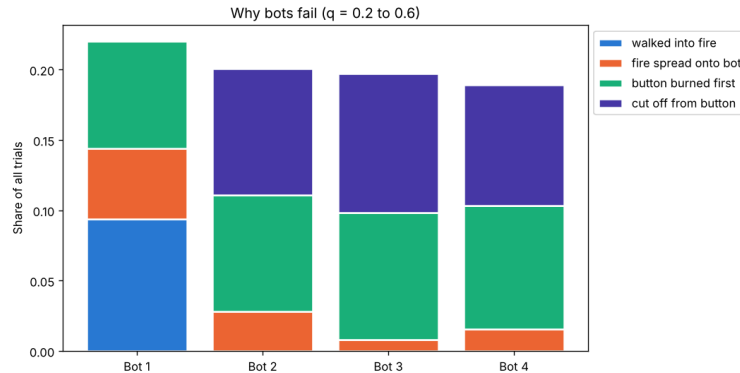


Figure 8: Why the bots fail, as a share of all trials at q from 0.2 to 0.6.

Since every bot faces the same fire, a trial that one bot lost and another won proves that a different sequence of moves would have saved the loser. That is true for 13.5% of Bot 1's failures, 6.1% of Bot 2's, 4.8% of Bot 3's and 2.3% of Bot 4's; for the rest, none of the four strategies found a way out. Head to head, Bot 4 lost 312 trials that Bot 3 won but won 754 that Bot 3 lost (and 247 against 931 for Bot 2). Figure 9 shows that its bad decisions aren't spread evenly. For q between 0.225 and 0.7, its good decisions outnumber its bad ones about 3 to 1 against Bot 3 and 4 to 1 against Bot 2. At $q \leq 0.2$ Bot 3 comes out ahead (69 trials to 56), and above 0.75 the two kinds of loss are about even. I looked at Bot 4's losses at both ends, and they come from two specific design choices.

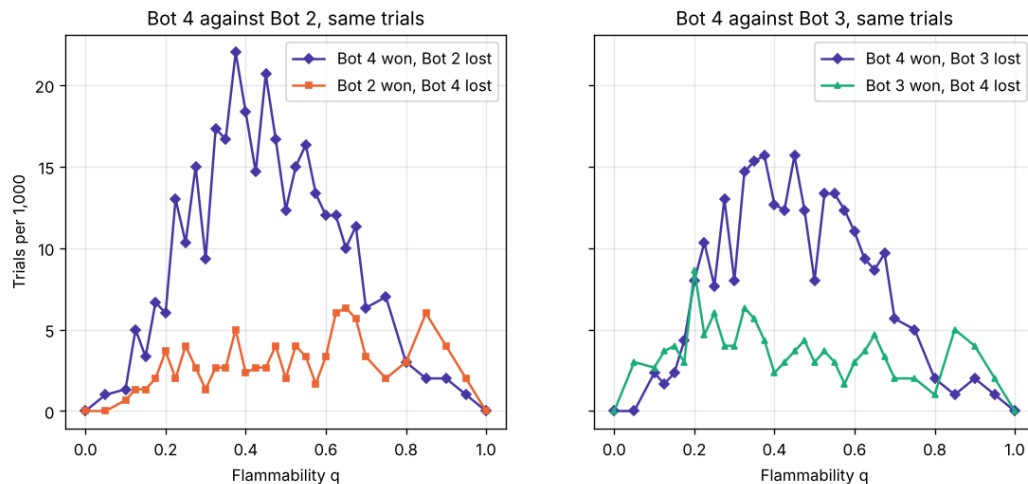


Figure 9: Out of every 1,000 trials at each q , how many Bot 4 won while Bot 2 (left) or Bot 3 (right) lost, and the reverse.

The first is at low q . In trial 2252 at $q = 0.2$ (Figure 10), the bot starts two cells from the fire and its shortest route runs right past it. By the race model, the first four cells on that route are 20%, 4%, 49% and 18% likely to be reached by the fire first. Each one is under the 60% threshold, so none of them is predicted fire, and Bot 4 takes the short route just like Bots 1 and 2; all three are caught on the very first step. Bot 3 keeps one cell away from the fire and wins in 64 steps. The problem is that a

threshold looks at one cell at a time, so several moderate risks in a row never add up to a reason to detour. A 59% risk costs nothing and a 61% risk costs 20 steps. A graded penalty, $20 \times P(\text{fire gets there first})$, would make small risks cost a little and let several of them add up.



Figure 10: Trial 2252 ($q = 0.2$). Bots 1, 2 and 4 take the short route past the fire and are caught at once; Bot 3 keeps its distance and wins in 64 steps.

The second is at very high q , and it is the known limitation from Section 1. In trial 96 at $q = 0.85$ (Figure 11), at move 21 the button is 5 steps away but one cell on the way is predicted fire, so Bot 4 picks a 17-step detour instead (cost 17, less than $5 + 20$). The detour's cells are scored using the bot's *shortest* distance to them, so the model doesn't notice that the bot will reach the button 12 steps later than it could have, and the fire, spreading at $q = 0.85$, catches the bot after 23 steps. Bots 2 and 3 went straight and won in 25. Fixing this means scoring each cell at the step the plan actually reaches it, so that slow detours pay for being slow, which requires searching over (cell, time) pairs instead of cells and costs more computation. Neither problem shows up in the middle range, where Bot 4 is clearly the best bot, but both are visible in the data at the ends.

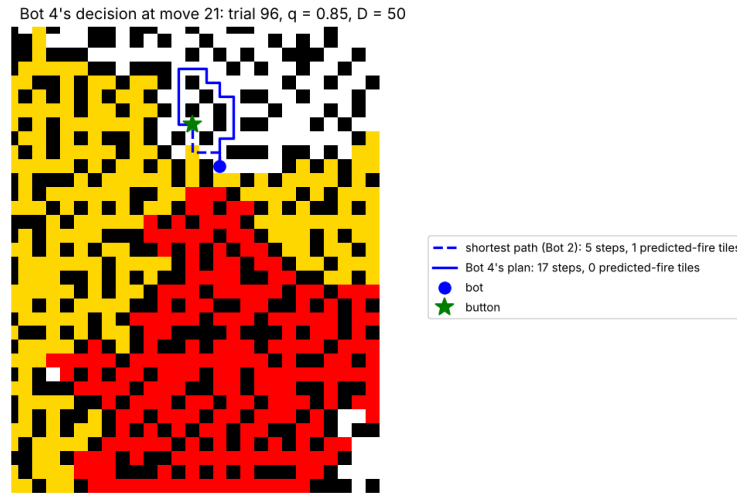


Figure 11: Bot 4's decision at move 21 of trial 96 ($q = 0.85$). The 5-step route to the button (dashed) crosses one predicted-fire cell (yellow), so Bot 4 takes the 17-step detour (solid) and is caught before it gets there.

Bot 4 didn't start out like this. My first version was more ambitious: a step-by-step forecast of the probability that every cell is burning at every future step, and an A* over (cell, time) pairs. It worked, but it was slow and I found it hard to explain clearly next to Bot 3, and the reader shouldn't have to dig through code to understand a bot. Adding the certain-win check showed me that half of all trials are decided before the fire even moves, which changed how I read the success curves, and counting the moves Bot 2's rule couldn't make showed me that Bot 4 mostly makes Bot 2's moves anyway, and that its value comes from the minority of moves where it doesn't. So I simplified Bot 4 to the race and a plain A* over cells: the same "who gets there first?" question with a closed-form answer. I then tuned it on 4,000 separate trials (500 at each q from 0.05 to 0.6, with a different seed), shown in Figure 12. Some of that went as I expected. The look-ahead is what matters: the literal one-step 60% rule did no better than Bot 2 (+0.07 points), exactly as my analysis of it predicted, while the race with a threshold of 0.6 and a penalty of 20 was 1.0 points ahead of Bot 2 and 0.7 ahead of Bot 3 on those trials. A high threshold (0.8) was worse, since it waits until the fire is nearly certain to arrive, and thresholds of 0.4 and 0.6 tied within the noise, so I kept 0.6, the number from my original idea. Other things surprised me. I expected the success curves to fall towards zero at high q , but they level off at the share of certain wins, and no bot ever lost one of those. The penalty barely mattered anywhere between 5 and 1000, because in these mazes a detour around predicted fire is usually either short or impossible. Bot 2 is better than I expected, given that Bot 4 agrees with it in 89% of trials. Bot 3's crude buffer actually beats Bot 4 at low q , because a fixed buffer is the right amount of caution when every individual risk is moderate but the bot passes many of them. And Bot 4 can be too clever at very high q , where its detours look safe to the model but aren't.

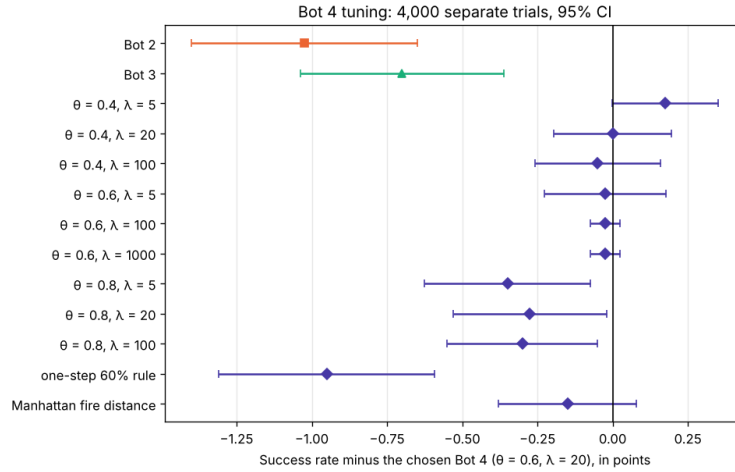


Figure 12: Tuning on 4,000 separate trials: each setting’s success rate minus the chosen Bot 4’s (threshold $\theta = 0.6$, penalty $\lambda = 20$) on the same trials, pooled over q , with 95% confidence intervals.

The ideal bot would maximise the probability of pressing the button. The fire is a Markov chain, so this is a Markov decision process whose state is the bot’s position plus the set of burning cells, and on a 50 by 50 ship that set has around 2^{1600} possible values, so the exact answer is out of reach. It can be approximated, though, from cheap to expensive. First, check for a certain win with two BFSs, and if a fireproof path exists, just follow it; no amount of extra thought can improve a guaranteed win. Second, predict the fire properly: instead of my race model’s single route, estimate the probability that each cell is burning t steps from now by simulating the fire forward many times from the current state (the fire is cheap to simulate), which accounts for the loops and multiple burning cells my model ignores. Third, plan over (cell, time) pairs, scoring each cell at the step the plan actually reaches it, for example with a cost of $-\ln(1 - p_t(c))$ per step so that a path’s total cost approximates $-\ln P(\text{surviving the path})$. That fixes both of Bot 4’s weaknesses: risks add up, and slow detours pay for being slow (my first Bot 4 was a version of this, and it was noticeably slower). Fourth, the bot replans after every step, so the value of a move really depends on the choices it will be able to make later. Looking ahead over its own future decisions, for example with Monte Carlo rollouts of both the fire and the bot, would capture things like preferring a cell from which two different routes reach the button.

All of that costs time, and in reality the fire would keep spreading while the bot thinks. Bot 4 already spends about 3 ms per move against Bot 2’s 0.5 ms, and the steps above would cost far more. My data say where that time is worth spending. When the outcome is already decided, thinking is wasted: in the certain-win trials (half of all trials) the shortest path wins anyway, and when the fire is between the bot and the button with no way around, nothing helps. At high q (0.8 and up) the bots are indistinguishable because the start decides everything, and Bot 4’s extra caution even hurts slightly, so the cheapest plan, the shortest path, is the intelligent choice. At low q (0.2 and below) cheap rules are as good as expensive ones, because the fire is slow enough that a simple buffer already avoids almost every danger. It is in between, roughly $0.2 < q < 0.7$, that better decisions pay off (Figure 9), and that is where more computation is worth its cost. So a practical ideal bot would be an “anytime” planner: always have the cheap BFS plan ready, only refine it when the situation is actually contested, and reuse work between steps (the fire only grows and the walls never change, so distance maps can be updated rather than recomputed from scratch).

Making a break for it is the intelligent choice in three situations. When a fireproof path exists, running isn’t even hope, it’s certainty. When the fire is so fast that every route is risky, waiting never helps: the fire only grows, so every cell only gets more dangerous with time, and the shortest route minimises how long the bot is exposed. That is also why none of my bots ever stays in place, since following any route later than necessary is never better than following it now. And when the race says the bot is well ahead of the fire on the shortest route, any detour just throws that lead away, so the right move is to run.